

Deep Learning — Assignment 3

Third assignment for the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Objectives

In this assignment you will

1. Experiment with convolutional neural networks
2. Train a convolutional neural network on a speech dataset
3. Investigate the effect of dropout and batch normalization
4. Define and train a residual neural network

Required software

If you haven't done so already, you will need to install the following additional libraries:

- `torch` and `torchvision` for PyTorch,
- `d2l`, the library that comes with [Dive into deep learning](#) book,
- `python_speech_features` to compute MFCC features.

All libraries can be installed with `pip install`.

```
In [ ]: %matplotlib inline
import os
import numpy as np
import matplotlib.pyplot as plt
from d2l import torch as d2l
import torch
from torch import nn
from scipy.io import wavfile

# Fix the seed, so outputs are exactly reproducible
torch.manual_seed(12345)
```

```
Out[ ]: <torch._C.Generator at 0x113022a10>
```

3.1 Convolution and receptive fields (9 points)

We will first define some helper functions to plot the receptive field of a node in a network.

```
In [ ]: def show_image(img, title=None, new_figure=True):
    if new_figure:
        plt.figure(figsize=(5, 5))
    im = plt.imshow(img, interpolation='none', aspect='equal', cmap='gray')
    ax = plt.gca();

    # plot pixel numbers and grid lines
    ax.set_xticks(np.arange(0, img.shape[1], 1))
    ax.set_yticks(np.arange(0, img.shape[0], 1))
    ax.set_xticklabels(np.arange(0, img.shape[1], 1))
    ax.set_yticklabels(np.arange(0, img.shape[0], 1))
    ax.set_xticks(np.arange(-.5, img.shape[1], 1), minor=True)
    ax.set_yticks(np.arange(-.5, img.shape[0], 1), minor=True)
    ax.grid(which='minor', color='gray', linestyle='-', linewidth=1.5)

    # hide axis outline
    for spine in ax.spines.values():
        spine.set_visible(False)

    if title is not None:
        plt.title(title)

    # set all weights in the network to one,
    # all biases to zero
    def fill_weights_with_ones(network):
        for name, param in network.named_parameters():
            if 'weight' in name:
                param.data = torch.ones_like(param.data)
            elif 'bias' in name:
                param.data = torch.zeros_like(param.data)
        return network

    def compute_receptive_field(network, input_size=(15, 15), binary=True):
        assert isinstance(network, torch.nn.Sequential), 'This only works with torch.nn
        for layer in network:
```

```

        if not isinstance(layer, (torch.nn.Conv2d, torch.nn.AvgPool2d)):
            raise Exception('Sorry, this visualisation only works for Conv2d and Av

# initialize weights to ones, biases to zeros
fill_weights_with_ones(network)

# find the number of input and output channels
input_channels = None
output_channels = None
for layer in network:
    if isinstance(layer, torch.nn.Conv2d):
        if input_channels is None:
            # first convolution layer
            input_channels = layer.in_channels
            output_channels = layer.out_channels
if input_channels is None:
    input_channels = 1

# first, we run the forward pass to compute the output shape give the input

# PyTorch expects input shape [samples, channels, rows, columns]
x = torch.zeros(1, input_channels, *input_size)
x.requires_grad = True

# forward pass: apply each layer in the network
y = x
y.retain_grad()
ys = [y]
for layer in network:
    y = layer(y)
    # keep track of the intermediate values so we can plot them later
    y.retain_grad()
    ys.append(y)

# second, we run the backward pass to compute the receptive field

# create gradient input: zeros everywhere, except for a single pixel
y_grad = torch.zeros_like(y)
# put a one somewhere in the middle of the output
y_grad[0, 0, (y_grad.shape[2] - 1) // 2, (y_grad.shape[3] - 1) // 2] = 1

# compute the gradients given this single one
y.backward(y_grad)

# receptive field is now in the gradient at each layer
receptive_fields = []
for y in ys:
    # the gradient for this layer shows us the receptive field
    receptive_field = y.grad
    if binary:
        receptive_field = receptive_field > 0
    receptive_fields.append(receptive_field)
return receptive_fields

def plot_receptive_field(network, input_size=(15, 15), binary=True):
    receptive_fields = compute_receptive_field(network, input_size, binary)

```

```

# plot the gradient at each layer
plt.figure(figsize=(4 * len(receptive_fields), 4))
for idx, receptive_field in enumerate(receptive_fields):
    plt.subplot(1, len(receptive_fields), idx + 1)
    # the last element of ys contains the output of the network
    if idx == len(receptive_fields) - 1:
        plot_title = 'output (%dx%d)' % (receptive_field.shape[2], receptive_fi
    else:
        plot_title = 'layer %d input (%dx%d)' % (idx, receptive_field.shape[2],
    # plot the image with the receptive field (sample 0, channel 0)
    show_image(receptive_field[0, 0], new_figure=False, title=plot_title)
    if not binary:
        plt.colorbar(fraction=0.047 * receptive_field.shape[0] / receptive_fiel

def receptive_field_size(network, input_size=(15, 15), binary=True):
    receptive_fields = compute_receptive_field(network, input_size, binary)
    return torch.count_nonzero(torch.flatten(receptive_fields[0][0,0]))

```

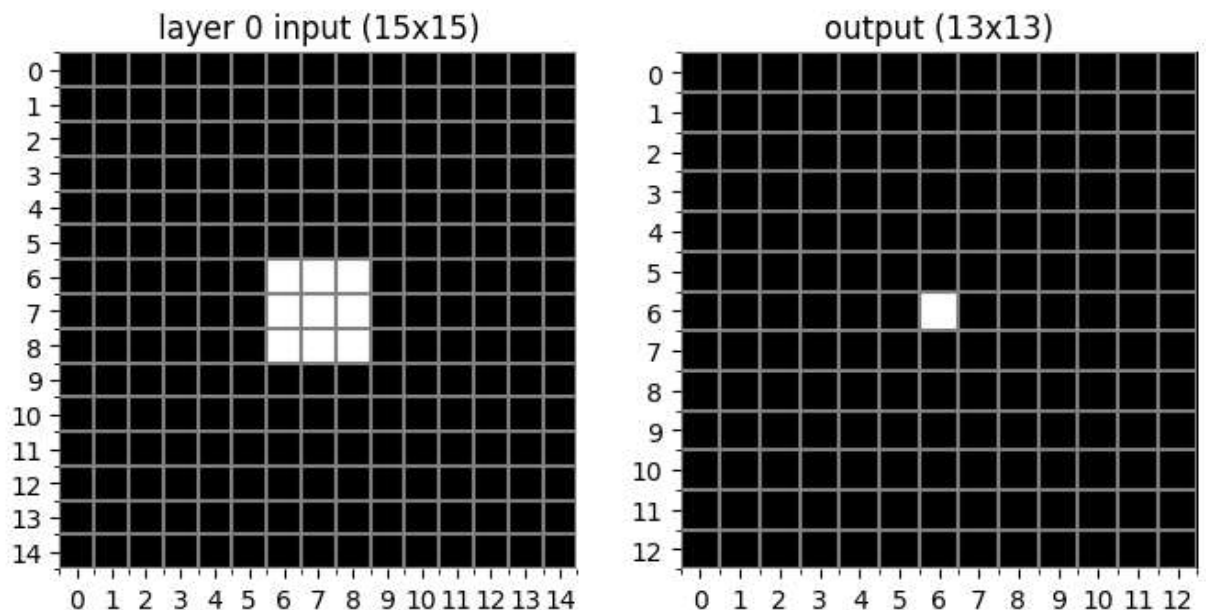
Using these functions, we can define a network and plot the receptive field of a pixel in the output.

(a) Run the code to define a network with one 3×3 convolution layer and plot the images.

```

In [ ]: net = torch.nn.Sequential(
        torch.nn.Conv2d(1, 1, kernel_size=(3, 3)),
    )
plot_receptive_field(net, input_size=(15, 15))

```



Read these images as follows:

- On the left, you see the input size of the network (here: 15 x 15 pixels) and the receptive field for one pixel in the output.

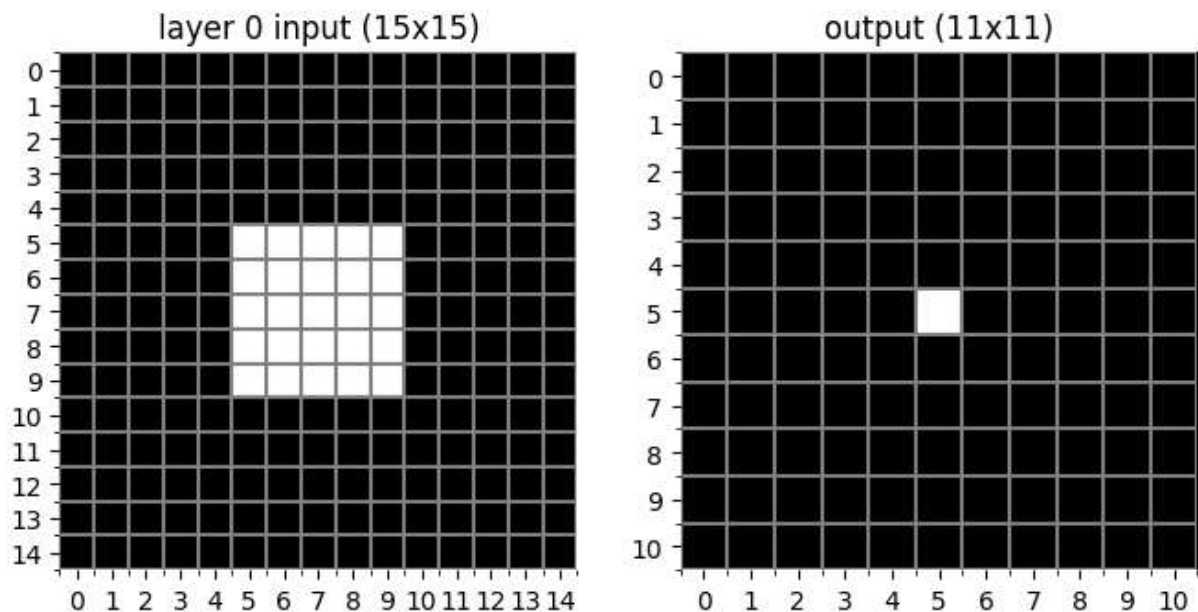
- On the right, you see the output size of the network (here: 13 x 13 pixels).

To visualize the receptive field of this network, we used the following procedure:

- We selected one pixel of the output (shown as the white pixel in the center in the image on the right).
- We computed the gradient for this pixel and plotted the gradient with respect to the input (the image on the left).
- This shows you the receptive field of the network: the output for the pixel we selected depends on these 9 pixels in the input.

(b) Use this method to plot the receptive field of a pixel in the output of a convolution layer with a kernel size of 5×5. (1 point)

```
In [ ]: # Plot the receptive field of a 5x5 convolution.
net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(5, 5)),
)
plot_receptive_field(net, input_size=(15, 15))
```



If you look at the result, you will see that two things have changed: the receptive field and the output size.

(c) How do the receptive field size and the output size depend on the kernel size? Give a formula. (1 point)

Receptive field size = kernel size [0] * kernel size [1]

Output size [0] = input size - kernel size [0] + 1

Output size [1] = input size - kernel size [1] + 1

Counting the number of parameters

In the previous question, you saw how the receptive fields of a 3x3 convolution differs from a 5x5 kernel convolution. But this is not the only difference: there is also a difference in the number of parameters in the network.

We can count the number of parameters in the network by computing the number of elements (e.g., the weights and biases in a convolution kernel) in the parameter list of the PyTorch network.

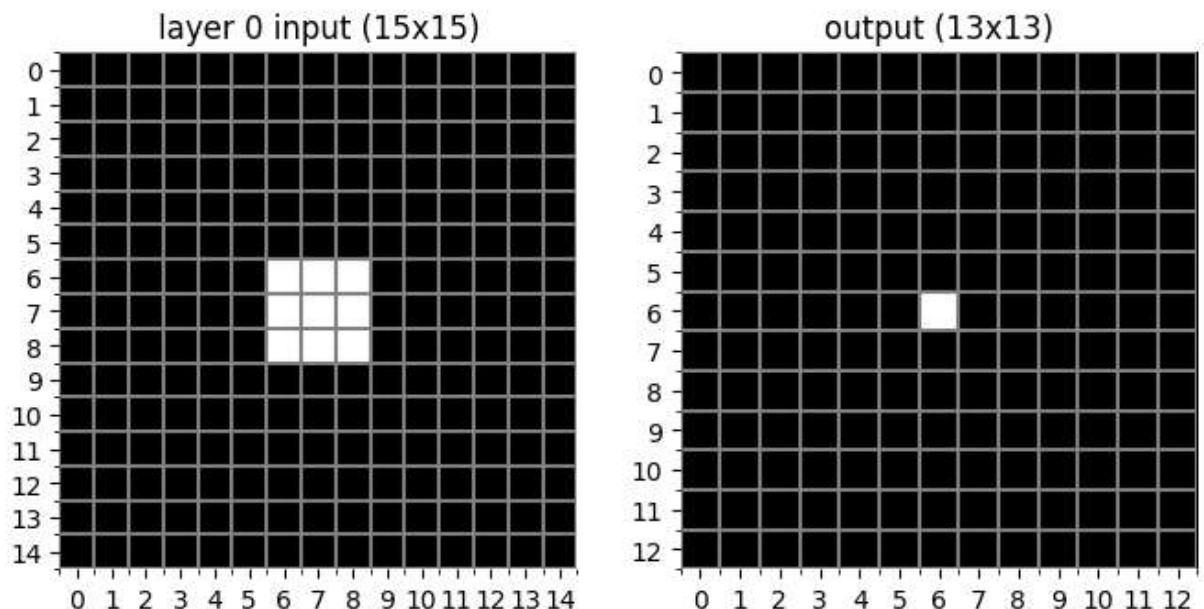
We'll define a small helper function to do this:

```
In [ ]: def print_parameter_count(network):  
        # sum the number of elements in each parameter of the network  
        count = sum([param.data.numel() for param in network.parameters()])  
        print('%d parameters' % count)
```

(d) Use the function to count the number of parameters for a 3x3 convolution.

```
In [ ]: net = torch.nn.Sequential(  
        torch.nn.Conv2d(1, 1, kernel_size=(3, 3)),  
    )  
    plot_receptive_field(net, input_size=(15, 15))  
    print_parameter_count(net)
```

10 parameters

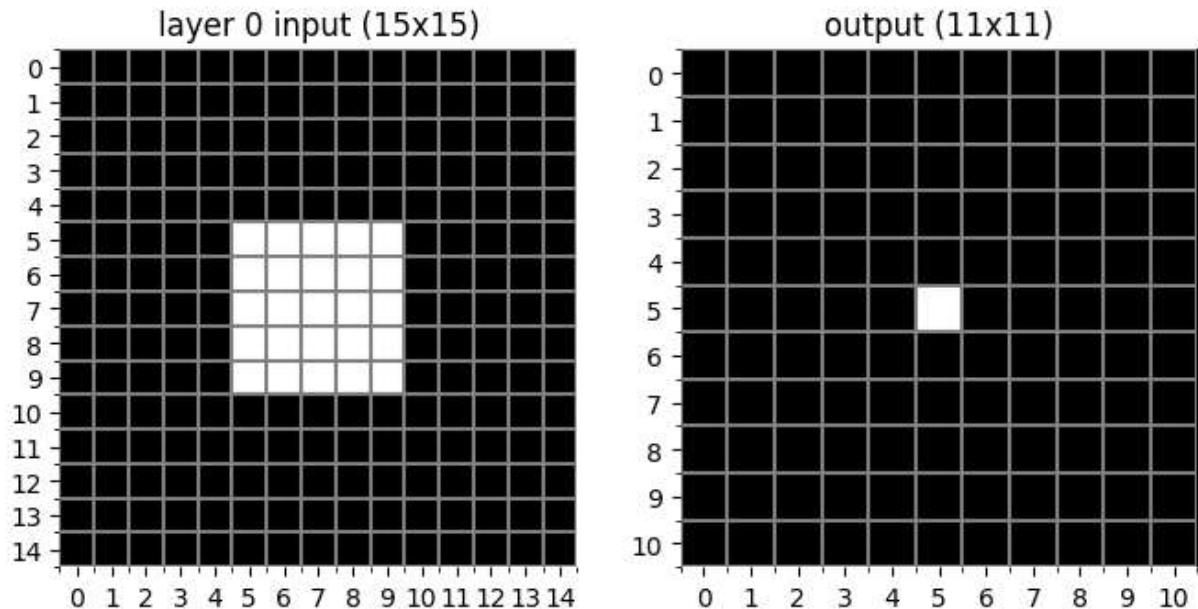


(e) Do the same to count the number of parameters for a 5x5 convolution. (1 point)

```
In [ ]: # count the number of parameters of a 5x5 convolution  
net = torch.nn.Sequential(  
    torch.nn.Conv2d(1, 1, kernel_size=(5, 5)),  
)
```

```
plot_receptive_field(net, input_size=(15, 15))
print_parameter_count(net)
```

26 parameters



(f) Explain the results by showing how to *compute* the number of parameters for the 3x3 and 5x5 convolutions. (1 point)

The number of parameters is the size of the kernel adding an extra parameter to take into account the bias term (when the input and output channels are 1).

Parameters = (kernel size[0] x kernel size[1]) + 1

kernel (3,3) -> 3 x 3 + 1 = 10

kernel (5,5) -> 5 x 5 + 1 = 26

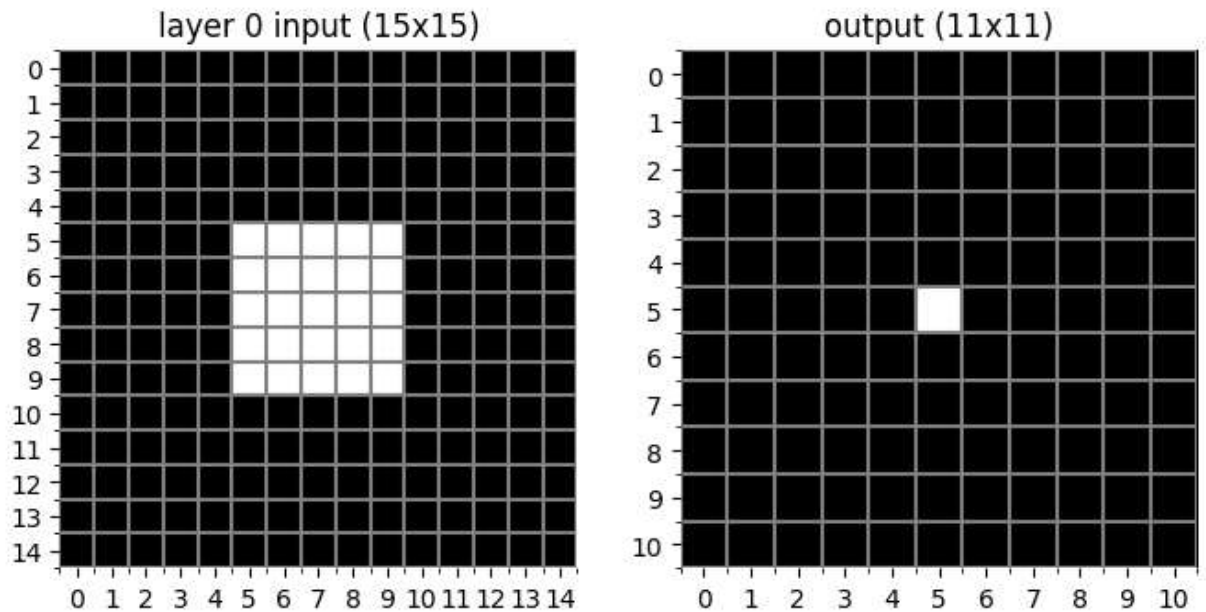
For these computations we used convolution layers with one input and one output channel.

We can also compute the results for a layer with a different number of channels.

(g) Define a network with a 5x5 convolution, 2 input channels and 3 output channels. Print the number of parameters.

```
In [ ]: # count the number of parameters of a 5x5 convolution with 2 input channels and 3 o
net = torch.nn.Sequential(
    torch.nn.Conv2d(2, 3, kernel_size=(5, 5)),
)
plot_receptive_field(net, input_size=(15, 15))
print_parameter_count(net)
```

153 parameters



(h) Show how to compute the number of parameters for this case. (1 point)

In this case, we need to take into account the number of channels. Therefore the computation would be:

$$(\text{kernel size}[0] \times \text{kernel size}[1] \times \text{input channels} \times \text{output channels}) + \text{output channels} = (5 \times 5 \times 2 \times 3) + 3 = 153$$

Preserving the size of the input image

The PyTorch documentation for `torch.nn.Conv2d` describes the parameters that you can use to define a convolutional layer. We will explore some of those parameters in the next questions.

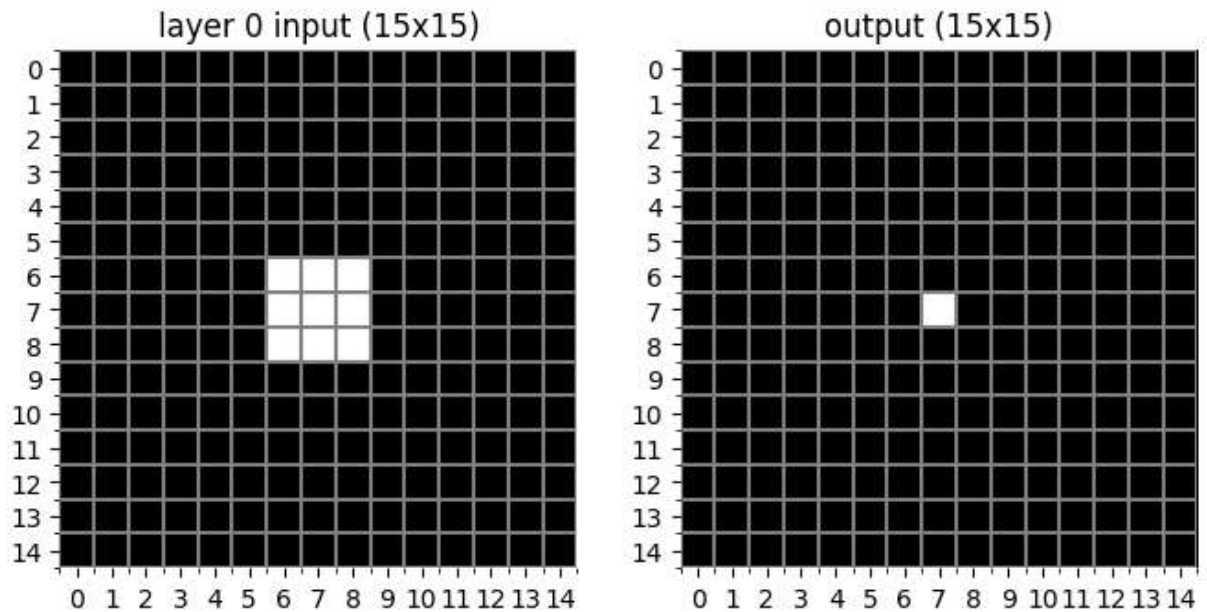
In the previous plot, you may have noticed that the output (11x11 pixels) was slightly smaller than the input (15x15 pixels).

(i) Define a network with a single 3x3 convolutional layer that produces an output that has the same size as the input. (1 point)

Use 1 input and 1 output channel.

```
In [ ]: # Define a network with a 3x3 kernel size that takes a 15x15 input image
#         and produces a 15x15 output image.

net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 'same'), # It is also valid
)
plot_receptive_field(net, input_size=(15, 15))
```



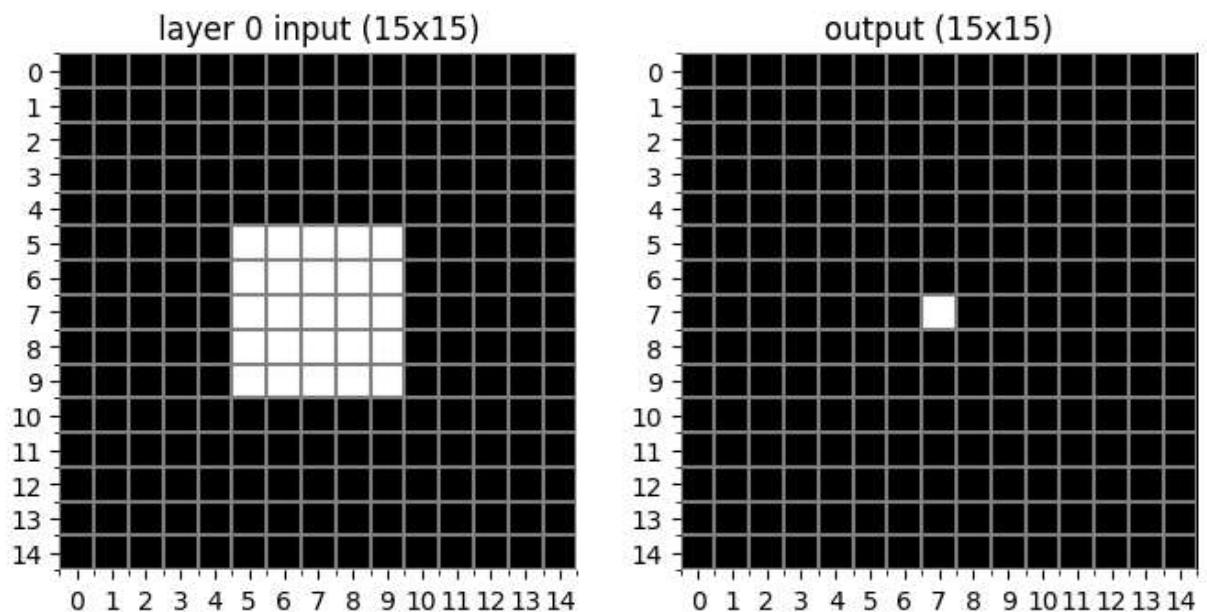
(j) Define a network with a single 5x5 convolutional layer that preserves the input size.

(1 point)

```
In [ ]: # Define a network with a 5x5 kernel size that takes a 15x15 input image
#        and produces a 15x15 output image.

net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(5, 5), padding = 'same'), # It is also valid
)
plot_receptive_field(net, input_size=(15, 15))
print_parameter_count(net)
```

26 parameters



Play around with some other values to see how this parameter behaves.

Multiple layers

As you have just seen, one way to increase the size of the receptive field is to use a larger convolution kernel. But another way is to use more than one convolution layer.

(k) Define a network with two 3x3 convolutions, preserving the image size. Show the receptive field and the number of parameters. (1 point)

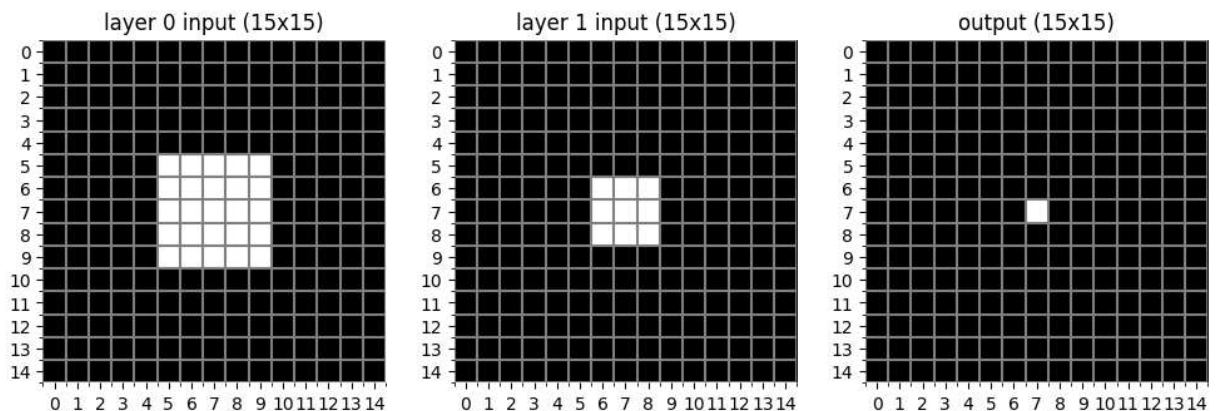
For this visualisation, do not use any activation functions, and use 1 channel everywhere.

```
In [ ]: # define a network with two 3x3 convolutions

net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 'same'), # It is also valid
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 'same'), # It is also valid
)

print(net)
plot_receptive_field(net, input_size=(15, 15))
print_parameter_count(net)
```

```
Sequential(
  (0): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (1): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=same)
)
20 parameters
```



Since we now have two layers, the visualization shows an extra image. From right to left, we have:

- Right: the output size and a single active pixel.
- Middle: the receptive field for the single output pixel between the first and second convolution.
- Left: the receptive field for the single output pixel in the input image.

We have now tried two ways to increase the receptive field size: increasing the kernel size, and using multiple layers.

(l) Compare the number of parameters required by the two options. Which one is more parameter-efficient? (1 point)

The Receptive Field when using a network with a convolution kernel (5, 5) is the same of using a network with two convolutional layers of kernel (3, 3).

The number of parameters of the 1st one is 26, whereas in the 2nd one we have 20 parameters. Therefore the network with two convolutional layers of kernel (3, 3) is more parameter efficient.

1st -> $(5 \times 5) + 1 = 26$

2nd -> $(3 \times 3) + 1 + (3 \times 3) + 1 = 20$

3.2 Variations on convolution (8 points)

Pooling

We can also increase the size of the receptive field by using a pooling layer.

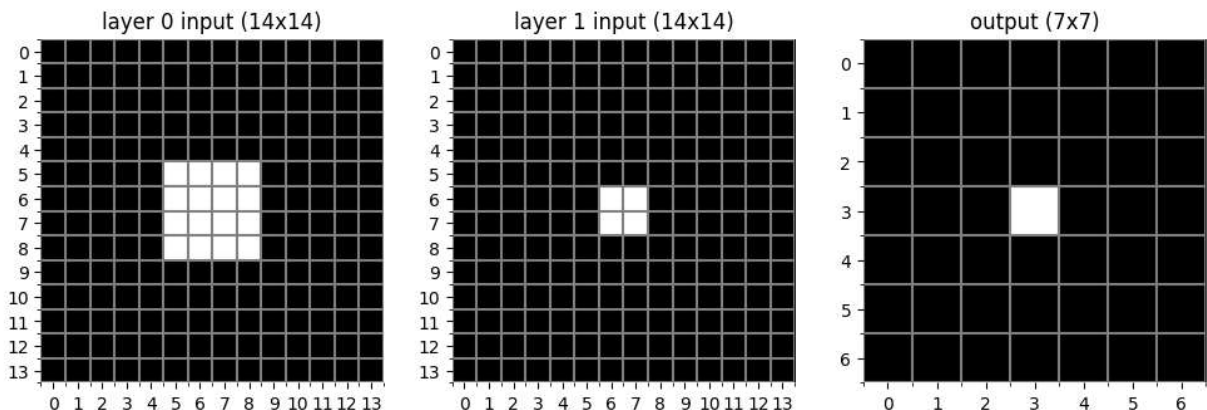
(a) Construct a network with a 3x3 convolution (preserving the input size) followed by a 2x2 average pooling. Plot the receptive field and print the number of parameters.

(1 point)

Use 1 input and 1 output channel.

```
In [ ]: # define a network with two 3x3 convolutions
net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 'same'), # It is also valid
    torch.nn.AvgPool2d(kernel_size=(2, 2))
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
```

```
Sequential(
  (0): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (1): AvgPool2d(kernel_size=(2, 2), stride=(2, 2), padding=0)
)
10 parameters
```



(b) Explain the number of parameters in this convolution + pooling network. (1 point)

For the 1st layer : $3 \times 3 + 1 = 10$

For the 2nd: it does not add parameters, just reduces the dimensionality

Dilation

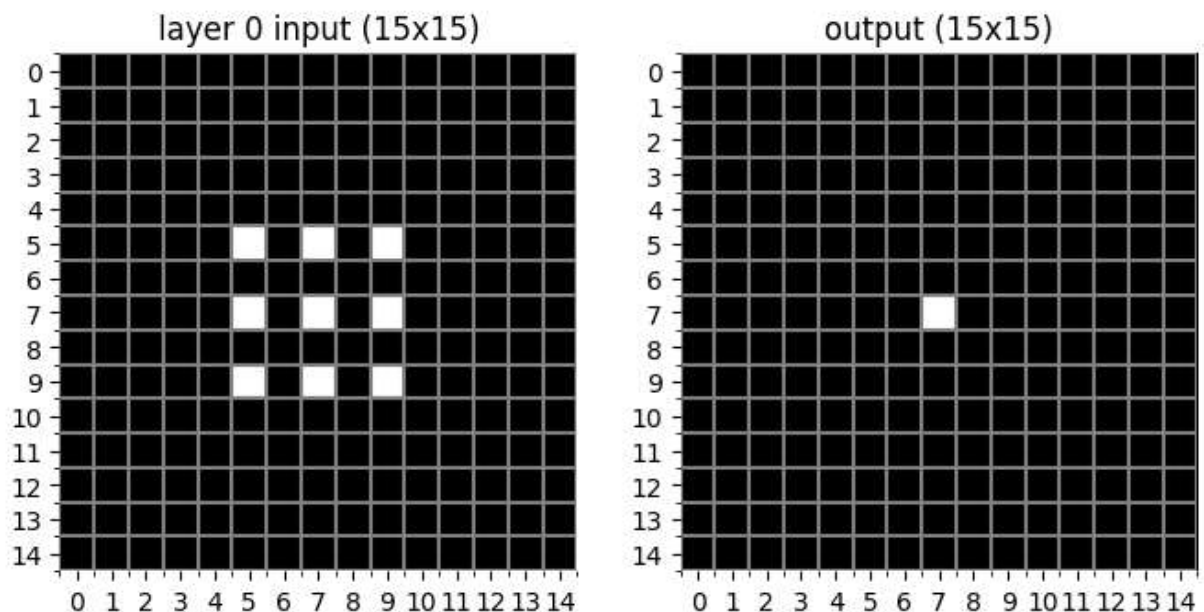
A third option to increase the receptive field is *dilation*.

(c) Define a network with 3x3 convolution with dilation that preserves the input size.

(1 point)

```
In [ ]: # define a network with one 3x3 convolution with dilation
net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 'same', dilation= 2), # It
)
# the output should also be 15x15 pixels
print(net)
plot_receptive_field(net, input_size=(15, 15))
print_parameter_count(net)
```

```
Sequential(
  (0): Conv2d(1, 1, kernel_size=(3, 3), stride=(1, 1), padding=same, dilation=(2,
2))
)
10 parameters
```



(d) Explain how dilation affects the receptive field.

(1 point)

Increases the receptive field due to the 'jumps' that result from it, but without increasing the number of parameters.

Using strides

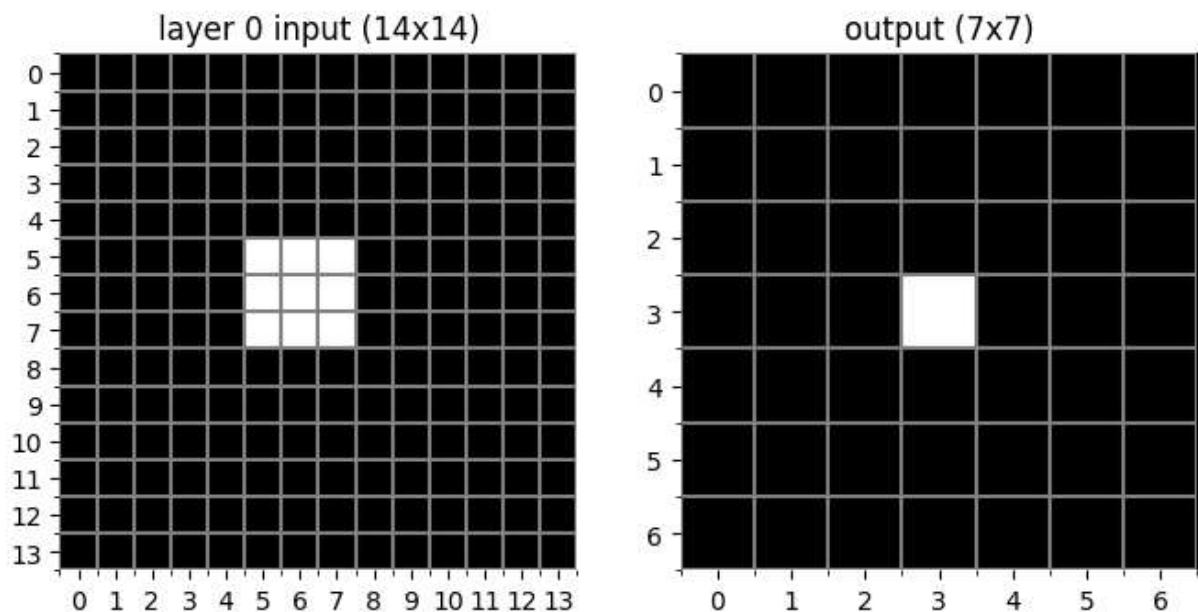
By default, convolution layers use a stride of 1.

(e) Change the network to use a stride of 2 and plot the result.

(1 point)

```
In [ ]: # increase the stride to 2
net = torch.nn.Sequential(
    torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding=(1, 1), stride = 2),
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
```

```
Sequential(
  (0): Conv2d(1, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
)
10 parameters
```



(f) Explain the new output size and compare the result with that of pooling. (1 point)

In this case, the output size is affected by a factor of $1/\text{stride}$. A stride of 2 gives us the same output size as a pooling of 2.

(g) Explain how the stride affects the receptive field of this single convolution layer.

(1 point)

The receptive field remains the same, but the 'jumps' for the computations would be of 2 steps instead of 1.

(h) Explain the number of parameters for this network.

(1 point)

$3 \times 3 + 1 = 10$. Stride does not add parameters.

3.3 Combining layers (7 points)

As you have seen, there are multiple ways to increase the receptive field. You can make interesting combinations by stacking multiple layers.

Let's try a few ways to make networks with a large receptive field. For each of the questions in this section:

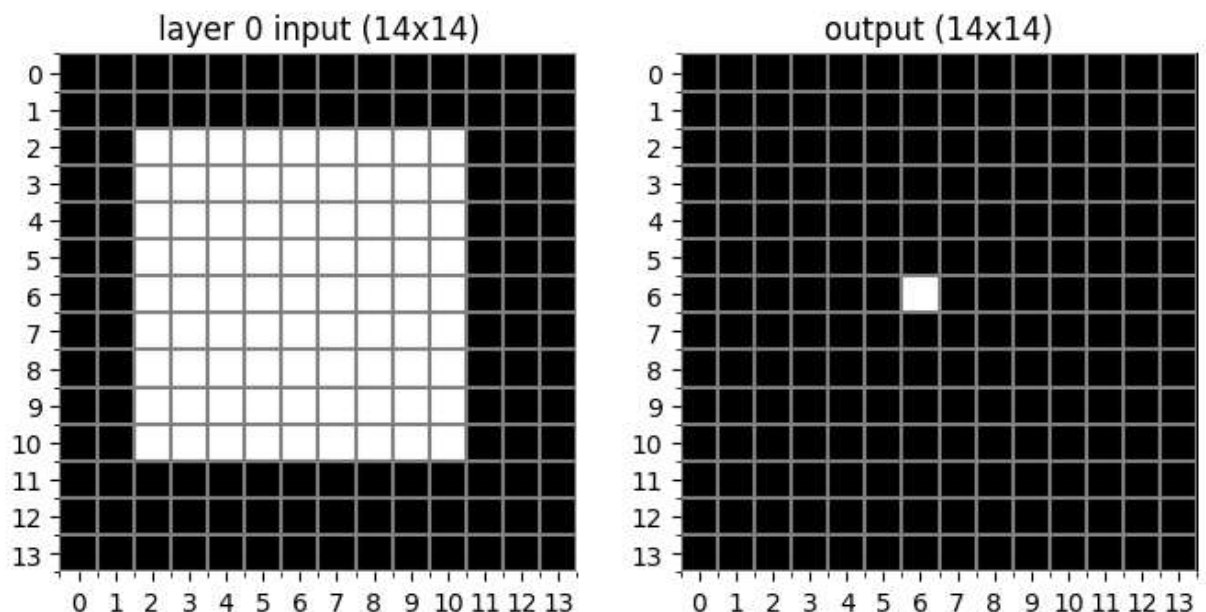
- Create a network where a pixel in the output has a 9x9 receptive field.
- Use 3 input channels and 3 output channels in every layer.
- In convolution layers, try to preserve the input size as much as possible.

(a) Make a network with a single convolution that satisfies the above conditions.

(1 point)

```
In [ ]: net = torch.nn.Sequential(
    torch.nn.Conv2d(3, 3, kernel_size=(9, 9), padding = 'same'), # It is also valid
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
assert receptive_field_size(net) == 9*9, "Receptive field of output pixel should be
```

```
Sequential(
  (0): Conv2d(3, 3, kernel_size=(9, 9), stride=(1, 1), padding=same)
)
732 parameters
```



Many popular network architectures use a sequence of 3x3 convolutions.

(b) Use only 3x3 convolutions.

(1 point)

```
In [ ]: net = torch.nn.Sequential(
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'), # It is also valid
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'), # It is also valid
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'), # It is also valid
```

```

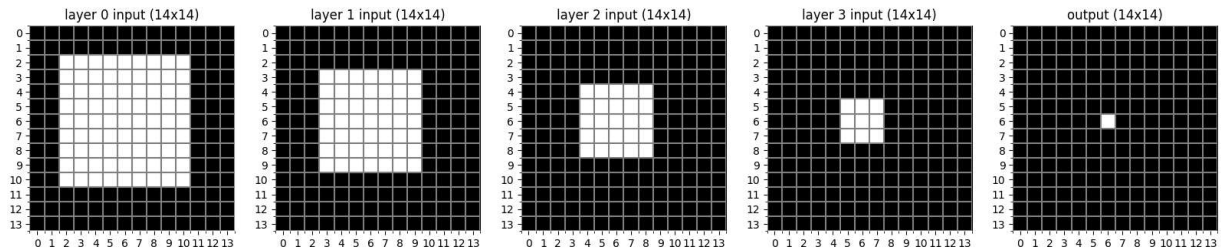
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'), # It is also valid
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
assert receptive_field_size(net) == 9*9, "Receptive field of output pixel should be

```

```

Sequential(
  (0): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (1): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (2): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (3): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
)
336 parameters

```



(c) Use a 2x2 average pooling layer in combination with one or more 3x3 convolutions.
(1 point)

```

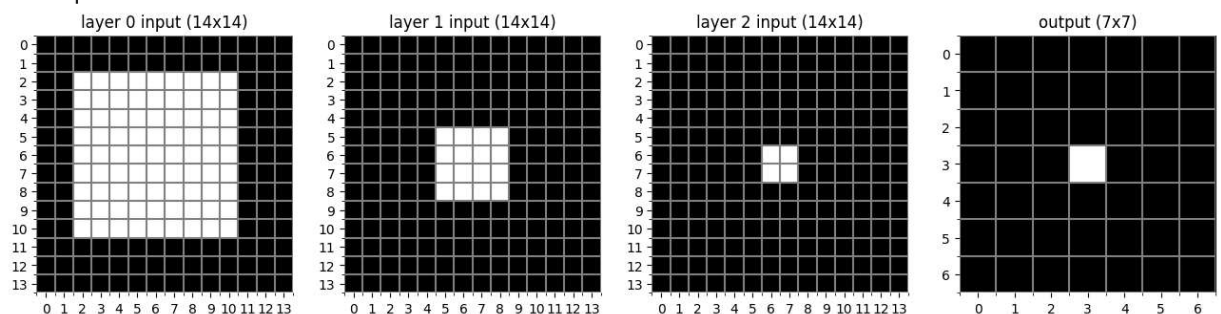
In [ ]: net = torch.nn.Sequential(
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 8, stride = 2),
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'), # It is also valid
    torch.nn.AvgPool2d(kernel_size=(2, 2))
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
assert receptive_field_size(net) == 9*9, "Receptive field of output pixel should be

```

```

Sequential(
  (0): Conv2d(3, 3, kernel_size=(3, 3), stride=(2, 2), padding=(8, 8))
  (1): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (2): AvgPool2d(kernel_size=(2, 2), stride=(2, 2), padding=0)
)
168 parameters

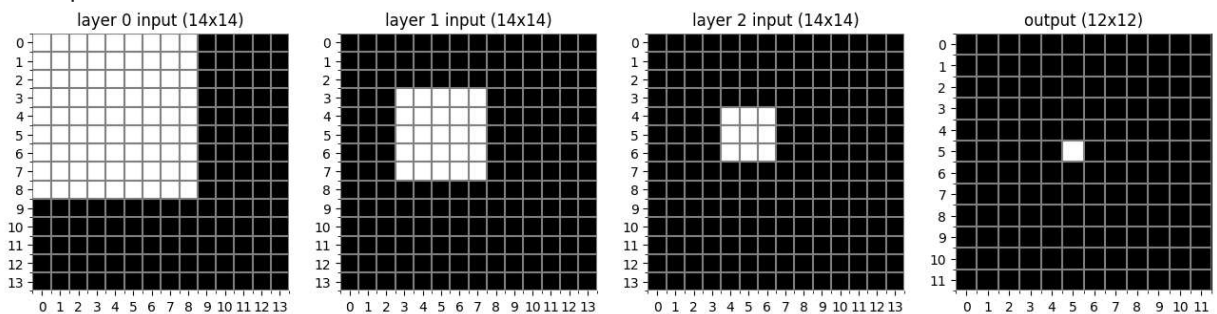
```



(d) Copy the previous convolution + pooling network and replace the pooling layer with a strided convolution layer.
(1 point)

```
In [ ]: net = torch.nn.Sequential(
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 8, stride = 2),
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same'),
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 11, stride = 3),
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
assert receptive_field_size(net) == 9*9, "Receptive field of output pixel should be
```

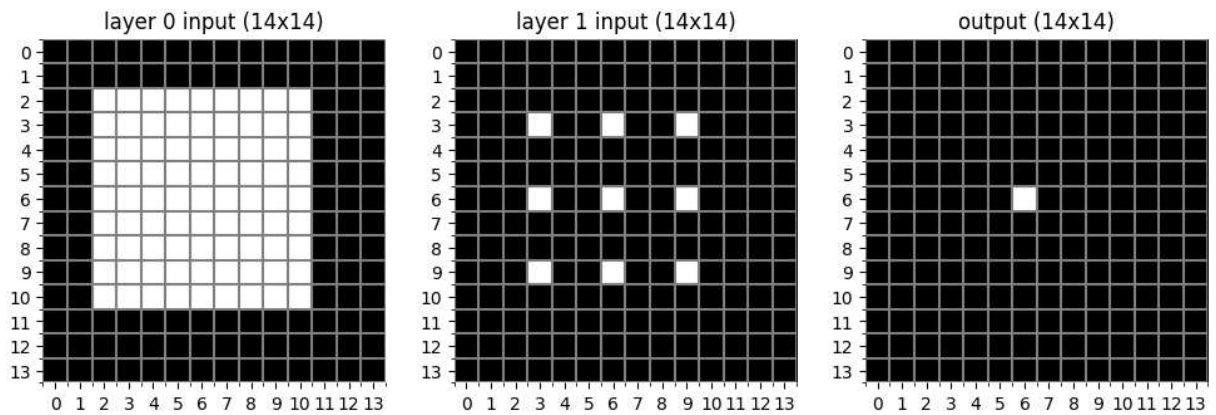
```
Sequential(
  (0): Conv2d(3, 3, kernel_size=(3, 3), stride=(2, 2), padding=(8, 8))
  (1): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same)
  (2): Conv2d(3, 3, kernel_size=(3, 3), stride=(3, 3), padding=(11, 11))
)
252 parameters
```



(e) Construct a network with exactly two 3x3 convolutions. Use dilation to get a receptive field of 9x9 pixels. (1 point)

```
In [ ]: net = torch.nn.Sequential(
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 1),
    torch.nn.Conv2d(3, 3, kernel_size=(3, 3), padding = 'same', dilation = 3)
)
print(net)
plot_receptive_field(net, input_size=(14, 14))
print_parameter_count(net)
assert receptive_field_size(net) == 9*9, "Receptive field of output pixel should be
```

```
Sequential(
  (0): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): Conv2d(3, 3, kernel_size=(3, 3), stride=(1, 1), padding=same, dilation=(3, 3))
)
168 parameters
```



(f) For each of the methods, list the number of layers, the number of parameters, and the size of the output of the network:

Method	Layers	Parameters	Output size
One 9x9 convolution	1	732	14x14
Many 3x3 convolutions	4	336	14x14
With pooling	3	168	7x7
With strided convolution	3	252	12x12
With dilation	2	168	14x14

(g) Compare the methods in terms of the number of parameters. (1 point)

As expected, the number of parameters increase with the dimension of the convolution kernel. Thus, in terms of the number of parameters, it is more efficient to add more layers instead of increasing the size of the kernel.

The network with pooling requires the least number of parameters, although it has the smaller output size too. Therefore, it is a low computational cost way of implementing downsampling. The NN with dilation has the same number of parameters, with a same input and output sizes.

When adding strides, the number of parameters increase slightly in comparison with the pooling one, with a larger output size, although shorter than the input size.

(h) Compare the methods in terms of the output size. How much downsampling do they do? (1 point)

The method that applies more downsampling is the one containing pooling. Strided also applies downsampling.

3.4 Padding in very deep networks (2 points)

Without padding, the output of a convolution is smaller than the input. This limits the depth of your network.

(a) How often can you apply a 3x3 convolution to a 15x15 input image?

```
In [ ]: # find the maximum number of layers
number_of_times = 25

# create a 15x15 input
x = torch.zeros(1, 1, 15, 15)
print('input size: %dx%d' % (x.shape[2], x.shape[3]))

# create a 3x3 convolution
conv = torch.nn.Conv2d(1, 1, kernel_size=(3, 3))

for n in range(number_of_times):
    # apply another convolution
    x = conv(x)
    print('layer %d, output size: %dx%d' % (n + 1, x.shape[2], x.shape[3]))
```

```
input size: 15x15
layer 1, output size: 13x13
layer 2, output size: 11x11
layer 3, output size: 9x9
layer 4, output size: 7x7
layer 5, output size: 5x5
layer 6, output size: 3x3
layer 7, output size: 1x1
```



```

-----
RuntimeError                                Traceback (most recent call last)
/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment 3/COPIAdl-assignment-3.ipynb Celada 72 line 1
      <a href='vscode-notebook-cell:/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment%203/COPIAdl-assignment-3.ipynb#Y131sZmlsZQ%3D%3D?line=8'>9</a> conv
= torch.nn.Conv2d(1, 1, kernel_size=(3, 3))
      <a href='vscode-notebook-cell:/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment%203/COPIAdl-assignment-3.ipynb#Y131sZmlsZQ%3D%3D?line=10'>11</a> for
n in range(number_of_times):
      <a href='vscode-notebook-cell:/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment%203/COPIAdl-assignment-3.ipynb#Y131sZmlsZQ%3D%3D?line=11'>12</a>
# apply another convolution
--> <a href='vscode-notebook-cell:/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment%203/COPIAdl-assignment-3.ipynb#Y131sZmlsZQ%3D%3D?line=12'>13</a>
x = conv(x)
      <a href='vscode-notebook-cell:/Users/carmengascogalvez/Desktop/2master/DeepLearning/Assignment%203/COPIAdl-assignment-3.ipynb#Y131sZmlsZQ%3D%3D?line=13'>14</a>
print('layer %d, output size: %dx%d' % (n + 1, x.shape[2], x.shape[3]))

File ~/anaconda3/lib/python3.11/site-packages/torch/nn/modules/module.py:1501, in Module._call_impl(self, *args, **kwargs)
    1496 # If we don't have any hooks, we want to skip the rest of the logic in
    1497 # this function, and just call forward.
    1498 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1499         or _global_backward_pre_hooks or _global_backward_hooks
    1500         or _global_forward_hooks or _global_forward_pre_hooks):
--> 1501     return forward_call(*args, **kwargs)
    1502 # Do not call functions when jit is used
    1503 full_backward_hooks, non_full_backward_hooks = [], []

File ~/anaconda3/lib/python3.11/site-packages/torch/nn/modules/conv.py:463, in Conv2d.forward(self, input)
    462 def forward(self, input: Tensor) -> Tensor:
--> 463     return self._conv_forward(input, self.weight, self.bias)

File ~/anaconda3/lib/python3.11/site-packages/torch/nn/modules/conv.py:459, in Conv2d._conv_forward(self, input, weight, bias)
    455 if self.padding_mode != 'zeros':
    456     return F.conv2d(F.pad(input, self._reversed_padding_repeated_twice, mode=self.padding_mode),
    457                     weight, bias, self.stride,
    458                     _pair(0), self.dilation, self.groups)
--> 459 return F.conv2d(input, weight, bias, self.stride,
    460                 self.padding, self.dilation, self.groups)

RuntimeError: Calculated padded input size per channel: (1 x 1). Kernel size: (3 x 3). Kernel size can't be greater than actual input size

```

Earlier in this assignment, you have used padding to address this problem. This seems ideal.

(b) Copy the previous code, add some padding, and show that we can now have an infinite number of layers.

(We are computer scientists and not mathematicians, so for the purpose of this question we'll consider 'infinite' to be equal to 25.)

```
In [ ]: # find the maximum number of layers
        number_of_times = 25

        # create a 15x15 input
        x = torch.zeros(1, 1, 15, 15)
        print('input size: %dx%d' % (x.shape[2], x.shape[3]))

        # create a 3x3 convolution
        conv = torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding = 1)

        for n in range(number_of_times):
            # apply another convolution
            x = conv(x)
            print('layer %d, output size: %dx%d' % (n + 1, x.shape[2], x.shape[3]))
```

```
input size: 15x15
layer 1, output size: 15x15
layer 2, output size: 15x15
layer 3, output size: 15x15
layer 4, output size: 15x15
layer 5, output size: 15x15
layer 6, output size: 15x15
layer 7, output size: 15x15
layer 8, output size: 15x15
layer 9, output size: 15x15
layer 10, output size: 15x15
layer 11, output size: 15x15
layer 12, output size: 15x15
layer 13, output size: 15x15
layer 14, output size: 15x15
layer 15, output size: 15x15
layer 16, output size: 15x15
layer 17, output size: 15x15
layer 18, output size: 15x15
layer 19, output size: 15x15
layer 20, output size: 15x15
layer 21, output size: 15x15
layer 22, output size: 15x15
layer 23, output size: 15x15
layer 24, output size: 15x15
layer 25, output size: 15x15
```

Without padding, we can only apply a 3x3 convolution 7 times. However, if we add a padding of 1, we can apply it infinite (25) times.

(c) Does it really work like this? Have a look at the following experiment.

- We simulate a convolution network with 25 convolution layers, with 3x3 kernels and the right amount of padding.
- We set the weights to 1/9 (so that the sum of the 3x3 kernel is equal to 1) and set the bias to zero.

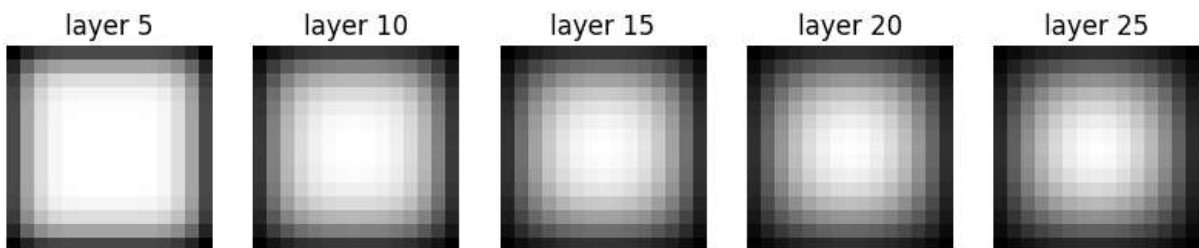
- We give this network a 15x15-pixel input filled with ones.
- We plot the output of layers 5, 10, 15, 20, and 25.

```
In [ ]: # create a 15x15 input filled with ones
x = torch.ones(1, 1, 15, 15)

# create a 3x3 convolution
conv = torch.nn.Conv2d(1, 1, kernel_size=(3, 3), padding=(1, 1))

# set weights to 1/9 (= sum to one), bias to zero
conv.weight.data = torch.ones_like(conv.weight.data) / 9
conv.bias.data = torch.zeros_like(conv.bias.data)

plt.figure(figsize=(10, 2))
for n in range(1, 26):
    # apply another convolution
    x = conv(x)
    # print('layer %d, output size: %dx%d' % (n + 1, x.shape[2], x.shape[3]))
    if n % 5 == 0:
        plt.subplot(1, 5, n // 5)
        plt.imshow(x[0, 0].detach().numpy(), cmap='gray')
        plt.axis('off')
        plt.title('layer %d' % n)
```



(d) Explain the pattern that we see in the output of the final layers. How does this happen, and what does this mean for our very deep networks? (2 points)

When we apply padding, we add 0s to our dataset so we increase its size without changing our original information. This is why we can observe an increasing 'black frame' in the output of the layers. This method allows us to use deeper nn without having size problems or losing information.

3.5 Spoken digits dataset (4 points)

Time for some practical experiments. The d2l book uses a dataset of images as a running example (FashionMNIST). In this assignment we will investigate CNNs in a completely different domain: speech recognition.

The dataset we use is the free spoken digits dataset, which can be found on <https://github.com/Jakobovski/free-spoken-digit-dataset>. This dataset consists of the digits 0 to 9, spoken by different speakers. The data comes as .wav files.

(a) Use the commands below (or a similar tool) to download the dataset. You can also use `git clone` to clone the repository mentioned above.

```
In [ ]: # ! mkdir -p free-spoken-digit-dataset
# ! wget -O - https://github.com/Jakobovski/free-spoken-digit-dataset/archive/refs/
```

```
In [ ]: # ! git clone https://github.com/Jakobovski/free-spoken-digit-dataset.git
```

Below is a function to load the data. We pad/truncate each sample to the same length. The raw audio is usually stored in 16 bit integers, with a range -32768 to 32767, where 0 represents no signal. Before using the data, it should be normalized. A common approach is to make sure that the data is between 0 and 1, between -1 and 1, or zero-mean unit-variance. Not all of these work well on this data, so later on, if your network doesn't seem to learn anything: try a different method to see if that works better.

(b) Update the below code to normalize the data to a reasonable range. (1 point)

```
In [ ]: samplerate = 8000

def normalize(v):
    norm = np.linalg.norm(v)
    mean = np.matrix.mean(np.matrix(v))
    std = np.matrix.std(np.matrix(v))
    if std == 0:
        return v
    return (v-mean)/std

def load_waveform(file, size = 6000):
    samplerate, waveform = wavfile.read(file)
    # Take first 6000 samples from waveform. With a samplerate of 8000 that corresp
    # Pad with 0s if the file is shorter
    waveform = np.pad(waveform,(0,size))[0:size]

    waveform_norm = normalize(waveform)
    return waveform_norm
```

The following code loads all .wav files in a directory, and makes it available in a pytorch dataset.

(c) Load the data into a variable `data` .

```
In [ ]: class SpokenDigits(torch.utils.data.Dataset):
    def __init__(self, data_dir):
        digits_x = []
        digits_y = []
        for file in os.listdir(data_dir):
            if file.endswith(".wav"):
                waveform = load_waveform(os.path.join(data_dir, file))
                label = int(file[0])
                digits_x.append(waveform)
                digits_y.append(label)
```

```

        # convert to torch tensors
        self.x = torch.from_numpy(np.array(digits_x, dtype=np.float32))
        # add an extra dimension to represent the "channels" (we start with 1 channel)
        self.x = self.x.unsqueeze(1)
        self.y = torch.from_numpy(np.array(digits_y))

    def __len__(self):
        return len(self.x)

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

data = SpokenDigits("free-spoken-digit-dataset/recordings/")

```

(d) Describe the dataset: how many samples are there, how many features does each sample have? How many classes are there? (1 point)

```
In [ ]: print(f'there are {len(data)} samples in the dataset, each with {data.x.shape[2]} f
```

there are 3000 samples in the dataset, each with 6000 features, belonging to 10 distinct classes

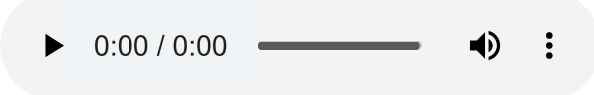
Here is code to play samples from the dataset to give you an idea what it "looks" like.

Note: If this step doesn't work in your notebook, then you can ignore it.

```
In [ ]: from IPython.display import Audio
def play(sample):
    print(f'Label: {sample[1]}')
    return Audio(sample[0][0].numpy(), rate=samplerate)
play(data[0])

```

Label: 5

Out []: 

Before continuing, we split the dataset into a training and a test set.

```
In [ ]: train_prop = 2/3
train_count = int(len(data) * train_prop)
train, test = torch.utils.data.random_split(data, [train_count, len(data) - train_c
```

The code above uses 2/3 of the data for training.

(e) Discuss an advantage and disadvantage of using more of the data for training. (2 points)

An advantage is that you will have more data to learn the model on, making it perform better. However, the downside to this is that you can test on less data, giving you less assurance of the model. Moreover, increased training data corresponds to increased computational cost.

Finally, we split the data into batches:

```
In [ ]: data_params = {'batch_size': 32}
train_iter = torch.utils.data.DataLoader(train, **data_params)
test_iter = torch.utils.data.DataLoader(test, **data_params)
```

3.6 One-dimensional convolutional neural network (8 points)

We will now define a network architecture. We will use a combination of convolutional layers and pooling. Note that we use 1d convolution and pooling here, instead of the 2d operations used for images.

(a) Complete the network architecture, look at the d2l book [chapter 7](#) and [chapter 8](#) for examples. (2 points)

```
In [ ]: def build_net():
    return torch.nn.Sequential(
        nn.Conv1d(1, 4, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),

        nn.Conv1d(4, 8, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),

        nn.Conv1d(8, 16, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),

        nn.Conv1d(16, 32, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),

        nn.Flatten(),
        nn.Linear(11872, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, 10))
```

(b) The first fully connected layer has input dimension 11872, where does that number come from? (1 point)

One of the waveforms has 6000 features; due to the convolutions and poolings, the output sizes will be condensed into:

conv 1: $6000 - 5 + 1 = 5996$

pool 1: $(5996 - 2) / 2 + 1 = 2998$

conv 2: $2998 - 5 + 1 = 2994$

pool 2: $(2994 - 2) / 2 + 1 = 1497$

conv 3: $1497 - 5 + 1 = 1493$

pool 3: $(1493 - 2) / 2 + 1 = 746$

conv 4: $746 - 5 + 1 = 742$

pool 4: $(742 - 2) / 2 + 1 = 371$

So the output size will be 371 before the fully connected layer, and there are 32 channels:
 $371 * 32 = 11872$

(c) How many parameters are there in the model? I.e. the total number of weights and biases. (1 point)

```
In [ ]: net = build_net()
        print('the model has')
        print_parameter_count(net)
```

the model has
1532090 parameters

(d) Suppose that instead of using convolutions, we had used only fully connected layers, while keeping the number of features on each hidden layer the same. How many parameters would be needed in that case approximately? (1 point)

Using fully connected layers instead of convolutional ones results in a surge in the parameter count:

For the initial convolutional layer with an input size of 6000×1 and an output size of $(6000 - 4) \times 4$, the equivalent fully connected layer would require $(3000 \times 1 + 1) \times 5996 \times 4 = 143,927,984$ parameters. Continuing the calculations for the subsequent layers, the total comes to 1,000,723,024 parameters - a 600x increase compared to the convolutional network.

The FashionMNIST dataset used in the book has 60000 training examples. How large is our training set? How would the difference affect the number of epochs that we need? Compare to [chapter 7.6](#) and [chapter 8.1](#) of the book.

(e) How many epochs do you think are needed? (1 point)

our data set is $(60000/3000 =)$ 20x smaller, so we should increase the epochs substantially to get to convergence; we should add a zero to the num_epochs.

```
In [ ]: lr, num_epochs = 0.02, 100
```

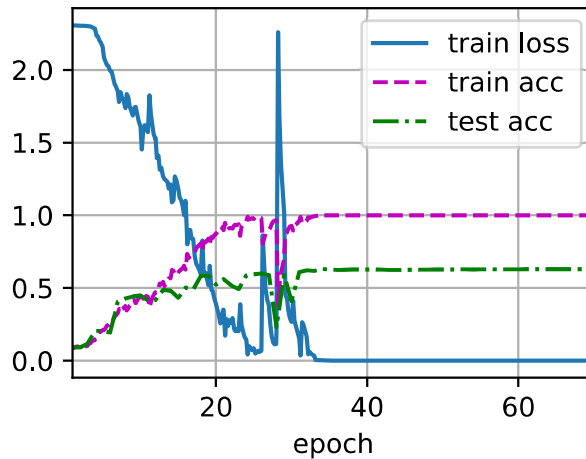
We will use the code from (a previous edition of) the d2l book to train the network. In particular, the `train` function, defined in [chapter 7.6](#). This function is reproduced below:

```
In [ ]: def train(net, train_iter, test_iter, num_epochs, lr, device = d2l.try_gpu()):
        """Train a model with a GPU (defined in Chapter 6)."""
        print('training on', device)
        net.to(device)
        optimizer = torch.optim.SGD(net.parameters(), lr=lr)
        loss = nn.CrossEntropyLoss()
        animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                                legend=['train loss', 'train acc', 'test acc'])
        timer, num_batches = d2l.Timer(), len(train_iter)
        for epoch in range(num_epochs):
            # Sum of training loss, sum of training accuracy, no. of examples
            metric = d2l.Accumulator(3)
            net.train()
            for i, (X, y) in enumerate(train_iter):
                timer.start()
                optimizer.zero_grad()
                X, y = X.to(device), y.to(device, torch.long)
                y_hat = net(X)
                l = loss(y_hat, y)
                l.backward()
                optimizer.step()
                with torch.no_grad():
                    metric.add(l * X.shape[0], d2l.accuracy(y_hat, y), X.shape[0])
                timer.stop()
                train_l = metric[0] / metric[2]
                train_acc = metric[1] / metric[2]
                if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                    animator.add(epoch + (i + 1) / num_batches,
                                  (train_l, train_acc, None))
            test_acc = d2l.evaluate_accuracy_gpu(net, test_iter)
            animator.add(epoch + 1, (None, None, test_acc))
        print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
              f'test acc {test_acc:.3f}')
        print(f'{metric[2] * num_epochs / timer.sum():.1f} examples/sec '
              f'on {str(device)}')
```

(f) Now train the network.

```
In [ ]: train(build_net(), train_iter, test_iter, num_epochs=70, lr=0.09)
```

```
loss 0.000, train acc 1.000, test acc 0.630
231.9 examples/sec on cpu
```



(g) Did the training converge?

(2 point)

If the training has not converged, maybe you need to change the number of epochs and/or the learning rate.

Hint: This is a non-trivial problem, so your network might take some time to learn. Don't give up too quickly, it might take 50-100 epochs before you see any significant changes in the loss curves.

TODO: Document the runs that you have performed and their results in the table below.

Experiment	epochs	lr	train accuracy	test accuracy	converged?
experiment 1	90	0.04	1.000	0.683	yes
experiment 2	75	0.08	1.000	0.699	yes
experiment 3	75	0.1	0.986	0.402	no
experiment 4	75	0.02	1.000	0.542	yes
experiment 5	120	0.009	1.000	0.737	yes
experiment 6	90	0.01	1.000	0.487	no
experiment 6	70	0.09	1.000	0.630	yes

3.7 Questions and evaluation (6 points)

(a) Does the network look like it is overfitting or underfitting? Explain how you see this.

(1 point)

It seems that the network is overfitting, since the training accuracy is much higher than the test accuracy, and we have achieved an almost perfect train loss. This is a sign that the network is not generalizing well to unseen data.

(b) Is what we have here a good classifier? Could it be used in a realistic application? Motivate your answer. (1 point)

The classifier that we obtained is not good enough to be used in a realistic application; the network is overfitting so the results won't be reliable enough.

(c) Do you think there is enough training data compared to the dimensions of the data and the number of parameters? Motivate your answer. (1 point)

It seems that there is not enough training data, since the network is overfitting. We have 60000 training examples, but the number of parameters is 11872, so we have 5x more parameters than training examples.

(d) How could the classifier be improved? Give at least 2 suggestions. (1 point)

- Adding more training data.
- Adding regularization techniques such as dropout or batch normalization. Both methods would decrease the overfitting of the network.

(e) The free spoken digits datasets has recordings from several different speakers. Is the test set accuracy a good measure of how well the trained network would perform for recognizing digits spoken by a new, unknown speaker? And if not, how could that be tested instead? (2 points)

As we did the train/test split randomly, the test set accuracy is not a good measure of how well the trained network would perform for recognizing digits spoken by a new, unknown speaker, as the chances of having the same speaker in both datasets are high.

This could be tested by splitting the data by speaker, and then using the test set accuracy as a measure of how well the trained network would perform for recognizing digits spoken by unknown speaker.

3.8 Variations (8 points)

One way in which the training might be improved is with dropout or with batch normalization.

(a) Make a copy of the network architecture from 3.6a below, and add dropout. (1 point)

Hint: see [chapter 8.1](#) for an example that uses dropout.

```
In [ ]: def build_net_dropout():  
        return torch.nn.Sequential(  
            nn.Conv1d(1, 4, kernel_size=5),  
            nn.ReLU(),
```

```

nn.AvgPool1d(kernel_size=2, stride=2),

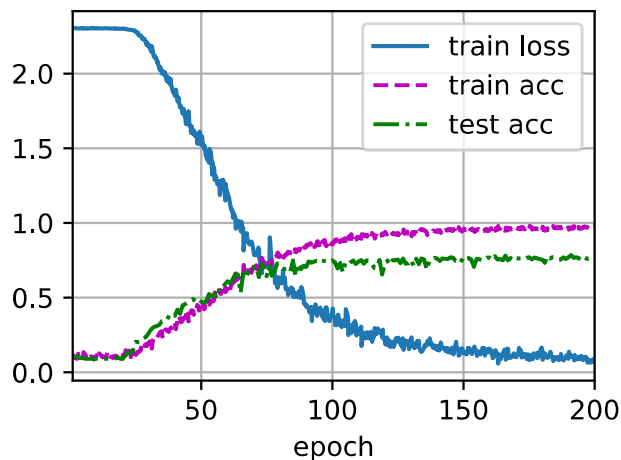
nn.Conv1d(4, 8, kernel_size=5),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),
nn.Conv1d(8, 16, kernel_size=5),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),
nn.Conv1d(16, 32, kernel_size=5),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),

nn.Flatten(),
nn.Linear(11872, 128), nn.ReLU(), nn.Dropout(p=0.5),
nn.Linear(128, 64), nn.ReLU(), nn.Dropout(p=0.5),
nn.Linear(64, 10))

train(build_net_dropout(), train_iter, test_iter, num_epochs=200, lr=0.02)

```

loss 0.082, train acc 0.975, test acc 0.781
258.6 examples/sec on cpu



(b) How does dropout change the results? Does this match what you saw on the simple network last week? (1 point)

Dropout is reducing the overfitting. However, the convergence is slower.

(c) Make a copy of the original network architecture, and add batch normalization to all convolutional and linear layers. (1 point)

Hint: see [chapter 8.5](#) for an example.

```

In [ ]: def build_net_batchnorm():
        return nn.Sequential(
            nn.Conv1d(1, 4, kernel_size=5),
            nn.BatchNorm1d(4),
            nn.ReLU(),
            nn.AvgPool1d(kernel_size=2, stride=2),

```

```

nn.Conv1d(4, 8, kernel_size=5),
nn.BatchNorm1d(8),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),

nn.Conv1d(8, 16, kernel_size=5),
nn.BatchNorm1d(16),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),

nn.Conv1d(16, 32, kernel_size=5),
nn.BatchNorm1d(32),
nn.ReLU(),
nn.AvgPool1d(kernel_size=2, stride=2),

nn.Flatten(),

nn.Linear(11872, 128),
nn.BatchNorm1d(128),
nn.ReLU(),

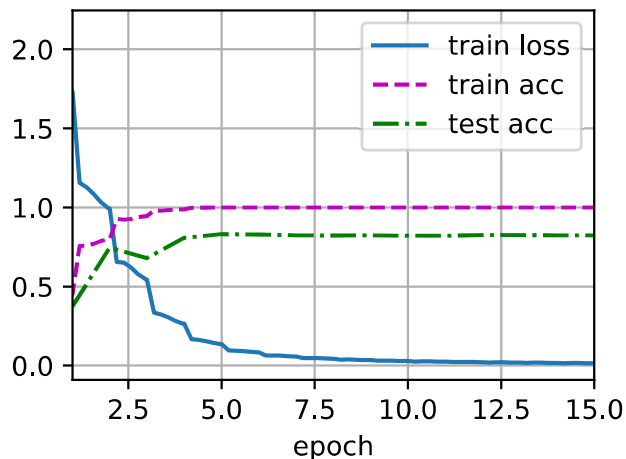
nn.Linear(128, 64),
nn.BatchNorm1d(64),
nn.ReLU(),

nn.Linear(64, 10)
)

train(build_net_batchnorm(), train_iter, test_iter, num_epochs=15, lr=0.02)

```

loss 0.016, train acc 1.000, test acc 0.824
223.6 examples/sec on cpu



(d) How does batch normalization change the results? Does this match what you saw on the simple network last week? (1 point)

It converges faster with batch normalization, and the accuracy is higher. It does not really match the simple network as the improvements for the simple network were marginal, whereas here there is a significant improvement.

Residual network

We can also try to use a residual network. The book has code for a 2d resnet in [Chapter 8.6](#).

(e) Copy the `Residual` module here, and adapt it for 1d convolutions. Use a kernel size of 5 for the convolution layers. (2 points)

Use residual blocks each containing two convolutional layers.

```
In [ ]: import torch
        from torch import nn
        from torch.nn import functional as F
        from d2l import torch as d2l

        class Residual(nn.Module):
            """The Residual block of ResNet."""
            def __init__(self, out_channels, use_1x1conv=False, strides=1):
                super().__init__()
                self.conv1 = nn.LazyConv1d(out_channels, kernel_size = 5,
                                             stride=strides, padding=1)

                self.conv2 = nn.LazyConv1d(out_channels, kernel_size = 5, padding=1)
                if use_1x1conv:
                    self.conv3 = nn.LazyConv1d(out_channels, kernel_size = 5,
                                                 stride=strides)
                else:
                    self.conv3 = None
                self.bn1 = nn.LazyBatchNorm1d()
                self.bn2 = nn.LazyBatchNorm1d()

            def forward(self, X):
                Y = F.relu(self.bn1(self.conv1(X)))
                Y = self.bn2(self.conv2(Y))
                if self.conv3:
                    X = self.conv3(X)
                Y += X
                return F.relu(Y)
```

(f) Make a copy of the network architecture from 3.6a, and replace the convolutions with residual blocks. (1 point)

```
In [ ]: resnet = torch.nn.Sequential(
        Residual(4, use_1x1conv=True),
        nn.AvgPool1d(kernel_size=2, stride=2),

        Residual(8, use_1x1conv=True),
        nn.AvgPool1d(kernel_size=2, stride=2),

        Residual(16, use_1x1conv=True),
        nn.AvgPool1d(kernel_size=2, stride=2),

        Residual(32, use_1x1conv=True),
```

```

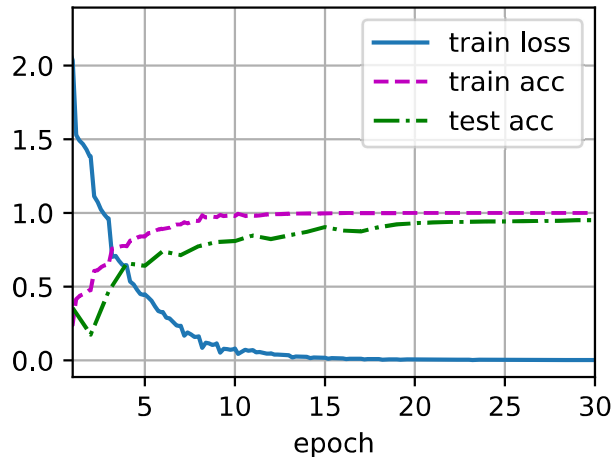
nn.AvgPool1d(kernel_size=2, stride=2),

nn.Flatten(),
nn.Linear(11872, 128), nn.ReLU(),
nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))

train(resnet, train_iter, test_iter, num_epochs= 30, lr=0.02)

```

loss 0.002, train acc 1.000, test acc 0.952
67.5 examples/sec on cpu



(g) How do residual connections change the results?

(1 point)

In this case, it takes longer to run (each residual blocks is made out of 3 convolutional layers). However, the accuracy is higher.

3.9 Feature extraction (5 points)

Given enough training data a deep neural network can learn to extract features from raw data like audio and images. However, in some cases it is still necessary to do manual feature extraction, in particular when working with smaller datasets like this one. For speech recognition, a popular class of features are [MFCCs](#).

Here is code to extract these features. You will need to install the `python_speech_features` first.

```

In [ ]: #!/pip install python_speech_features
from python_speech_features import mfcc

def load_waveform_mfcc(file, size = 6000):
    samplerate, waveform = wavfile.read(file)
    waveform = np.pad(waveform, (0, size))[0:size] / 32768
    return np.transpose(mfcc(waveform, samplerate))

```

(a) Implement a variation of the dataset that uses these features.

(2 points)

```
In [ ]: class SpokenDigitsMFCC(torch.utils.data.Dataset):
    def __init__(self, data_dir):
        digits_x = []
        digits_y = []
        for file in os.listdir(data_dir):
            if file.endswith(".wav"):
                waveform = load_waveform_mfcc(os.path.join(data_dir, file))
                label = int(file[0])
                digits_x.append(waveform)
                digits_y.append(label)

        self.x = torch.from_numpy(np.array(digits_x, dtype=np.float32))
        self.y = torch.from_numpy(np.array(digits_y))
    def __len__(self):
        return len(self.x)
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

data_mfcc = SpokenDigitsMFCC('free-spoken-digit-dataset/recordings/')
train_count_mfcc = int(len(data_mfcc) * train_prop)
train_mfcc, test_mfcc = torch.utils.data.random_split(data_mfcc, [train_count_mfcc,
train_iter_mfcc = torch.utils.data.DataLoader(train_mfcc, **data_params)
test_iter_mfcc = torch.utils.data.DataLoader(test_mfcc, **data_params)

assert next(iter(train_iter_mfcc))[0].shape == torch.Size([data_params['batch_size']
```

The MFCC features will have 13 channels instead of 1 (the `unsqueeze` operation is not needed).

(b) Inspect the shape of the data, and define a new network architecture that accepts data with this shape. (1 point)

```
In [ ]: number_of_samples = len(data_mfcc)
data_shape = data_mfcc[0][0].shape
number_of_classes = len(np.unique([y for (x,y) in data_mfcc]))
print('Number of samples:', number_of_samples)
print('Shape of samples:', data_shape)
print('Number of classes:', number_of_classes)
```

```
Number of samples: 3000
Shape of samples: torch.Size([13, 74])
Number of classes: 10
```

```
In [ ]: def build_net_mfcc():
    return torch.nn.Sequential(

        nn.Conv1d(13, 32, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),

        nn.Conv1d(32, 64, kernel_size=5),
        nn.ReLU(),
        nn.AvgPool1d(kernel_size=2, stride=2),
```

```
nn.Conv1d(64, 128, kernel_size=5),
nn.ReLU(),
nn.Flatten(),

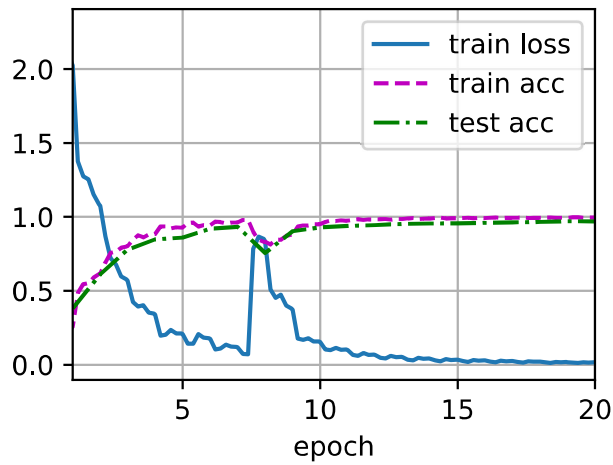
nn.Linear(1408, 128), nn.ReLU(),
nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))
```

(c) Train the network with the MFCC features.

(1 point)

```
In [ ]: train(build_net_mfcc(), train_iter_mfcc, test_iter_mfcc, num_epochs=20, lr=0.02)
```

loss 0.016, train acc 0.996, test acc 0.969
1737.3 examples/sec on cpu



(d) What would be needed to get a fully neural network approach to work as well as MFCC features?

(1 point)

Using a different architecture. After seeing the performance obtained with resnet and the improvement in the performance with BN, we could try to use a deeper network with more layers and residual blocks.

Using a pre-trained network could also help, as it would have already learned some features.

The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 57 points.

Version 6717cb8 / 2023-09-15